Task 1: Interactive Quote Generator with Statistics

Create a motivational quote application that displays random programming quotes and tracks user interactions.

Requirements:

1. Create an array of 12 programming/learning quotes (each quote should be at least 15 characters)
2. Display a random quote when the user clicks a "Get Inspired" button
3. Display the following statistics about the current quote:
    - Total character count (including spaces)
    - Word count (hint: think about splitting)
    - Number of vowels (a, e, i, o, u - case insensitive)
4. Add a "Favorite" button that stores the quote in a separate favorites array
5. Display all favorite quotes when user clicks "Show Favorites"

Questions:

- What happens if your quotes array is empty? How would you handle this?
- Why is it important to use Math.floor() with Math.random() for array indexing?
- How would you ensure the same quote doesn't appear twice in a row?

Task 2: Multi-Format Date & Time Display System

Build a comprehensive date-time display system that shows time in multiple formats and calculates time differences.

Requirements:

1. Create a button "Show Current Date/Time" that displays:
    - Current date in format: "Monday, January 11, 2026"
    - Current time in 12-hour format with AM/PM
    - Current time in 24-hour format
    - The current day of the week
    - Days remaining until the end of the current month
2. Add an "Age Calculator" section:
    - Ask user to enter their birth date (DD-MM-YYYY)
    - Calculate and display their exact age in years, months, and days
    - Display which day of the week they were born on
3. Create a "Countdown Timer" that shows:
    - Days, hours, minutes until New Year 2027

Task 3: Advanced Email & Username Validator (String Methods - No RegExp)

Create a comprehensive validation system using ONLY string methods (charAt, indexOf, substring, slice, etc.)

Requirements:

Part A - Email Validation:

1. Ask user to enter their email address
2. Validate the following rules WITHOUT using RegExp:
    - Contains exactly one @ symbol
    - @ is not at the first or last position
    - Contains at least one dot (.) after the @ symbol
    - The dot is not immediately after @ (no "@.")
    - There are at least 2 characters before @
    - There are at least 2 characters between @ and the last dot
    - There are at least 2 characters after the last dot
3. Display specific error messages for each failed validation

Part B - Username Validation:

1. Ask user to create a username
2. Validate:
    - Length between 5 and 15 characters
    - Starts with a letter (not a number or special character)
    - Contains no spaces
    - Extract and display the first 3 characters as a "user code"

Questions:

- How can you find the last occurrence of a character in a string using only indexOf()?
- Why might indexOf() returning -1 be useful for validation?
- What's the difference between checking string.length === 0 vs string === ""?

Task 5: Advanced Student Records System

Create a comprehensive student grade analysis system with sorting, filtering, and statistical calculations.

Given Initial Data:let grades = [78, 95, 45, 32, 88, 100, 55, 73, 91, 18, 67, 82];

Requirements:

1. **Basic Operations:**
   - Sort grades in descending order
   - Find the highest valid grade (≤ 100) using find()
   - Find the lowest grade using appropriate method
   - Calculate and display the average grade
2. **Filtering Operations:**
   - Display all failing grades (< 60) using filter()
   - Display all excellent grades (≥ 90) using filter()
   - Count how many students passed (≥ 60)
3. **Grade Categories:** Create a function that categorizes all grades into:
   - Excellent: 90-100
   - Very Good: 80-89
   - Good: 70-79
   - Pass: 60-69
   - Fail: below 60

Display the count of students in each category.

4. **Grade Adjustment:**
   - Add a "Curve Grades" button that adds 5 points to all grades below 60
   - Ensure no grade exceeds 100 after curving
   - Display the updated grades

Questions:

- What's the difference between sort() and sort((a,b) => a - b)? Why does it matter for numbers?
- Why does find() stop at the first match? When would you use filter() instead?
- How would you remove duplicate grades from the array while maintaining order?

Task 4: Student Grade Management System

Build a complete student management system with complex operations on arrays of objects.

Initial Data Structure:

```
let students = [
  { name: "Ahmed Ali", degree: 87, major: "CS" },
  { name: "Fatma Hassan", degree: 92, major: "IT" },
  { name: "Omar Khaled", degree: 45, major: "CS" },
  { name: "Layla Mohamed", degree: 78, major: "IS" },
  { name: "Youssef Ibrahim", degree: 56, major: "IT" },
  { name: "Nour Ahmed", degree: 95, major: "CS" }
];
```

Requirements:

1. **Search Operations:**
    - Find and display the first student with a degree between 90-100 (use find())
    - Find and display all CS major students with degrees above 80
    - Find the student with the exact name entered by user (case-insensitive search)
2. **Display Operations:**
    - Display names of all students who failed (degree < 60) using filter()
    - Display all students sorted alphabetically by name
    - Display students sorted by degree (highest first)
3. **Modification Operations:**
    - Add a new student using push() with user input (name, degree, major)
    - Remove the last student using pop() and display who was removed
    - Use splice() to add 2 new students after the 2nd position
    - Use splice() to remove the 4th student from the array
4. **Iteration Demonstrations:**
    - Use for...in loop to display all array indices and their corresponding student names
    - Use for...of loop to display a formatted list of all students
5. **Advanced Statistics:**
    - Calculate average degree for each major (CS, IT, IS)
    - Find which major has the highest average
    - Count students in each major

Task 6: Birthday Date Validator & Calculator

Create a sophisticated birth date validation and processing system using string manipulation and Date objects.

Requirements:

1. **Input & Validation:**
    - Create a prompt asking user to enter birth date in format: DD-MM-YYYY (e.g., 15-03-1998)

- Validate using ONLY string methods (no RegExp):
  - String length is exactly 10 characters
  - Contains exactly 2 hyphens (-)
  - Hyphens are at positions 2 and 5 (indices 2 and 5)
  - DD is between 01-31
  - MM is between 01-12
  - YYYY is between 1900-2024
  - All characters except hyphens are digits

2. **Date Processing (if valid):**
   - Extract day, month, year using string methods
   - Create a Date object with these values
   - Display:
     - The date in full text format (e.g., "Tuesday, March 15, 1998")
     - User's exact age in years, months, and days
     - Day of the week they were born
     - Their next birthday date and day of week
     - Days until their next birthday
3. **Error Handling:**
   - If format is wrong: "Error: Please use DD-MM-YYYY format"
   - If date components are invalid: "Error: Invalid date values"
   - If date is in the future: "Error: Birth date cannot be in the future"
4. **Additional Features:**
   - Add a "Calculate Age" button that triggers the validation function
   - Display results in a nicely formatted div (not alerts)
   - Add a "Reset" button to clear all inputs and results

Task 7: Text Analysis & Transformation Tool

Build a comprehensive text analysis tool that demonstrates mastery of string methods.

Requirements:

1. **Text Input:**
   - Provide a textarea for user to input any text (minimum 50 characters)
2. **Analysis Features - Display:**

    - Total characters (including spaces)
    - Total characters (excluding spaces)
    - Total words
    - Total sentences (count periods, exclamation marks, question marks)
    - Total paragraphs (count by line breaks)
    - Longest word in the text
    - Shortest word in the text
    - Most frequently used word (case-insensitive)
3. **Transformation Features:**
    - Button "Convert to Uppercase" - converts entire text
    - Button "Convert to Lowercase" - converts entire text
    - Button "Title Case" - capitalizes first letter of each word
    - Button "Reverse Text" - reverses the entire text
    - Button "Remove Extra Spaces" - replaces multiple spaces with single space
4. **Search & Replace:**
    - Input field to search for a word
    - Display how many times the word appears (case-insensitive)
    - Highlight all occurrences in the text
    - Input field to replace all occurrences with a new word

Questions:

- How can you count words accurately when there are multiple spaces between words?
- What's the most efficient way to find the most frequently used word?
- How do you preserve the original text while showing transformations?

Task 8: Error Handling - Calculator

Build a calculator that demonstrates comprehensive error handling techniques.

Requirements:

1. **Basic Calculator Operations:**
    - Addition, Subtraction, Multiplication, Division, Power (x^y), Square Root
    - Input: Two numbers and an operation
2. **Implement Error Handling for:**

- **Input Validation Errors:**
  - Non-numeric input → throw Error "Invalid input: Please enter numbers only"
  - Empty input → throw Error "Empty input: Both fields are required"
  - Special characters → throw Error "Invalid characters detected"
- **Operation Errors:**
  - Division by zero → throw Error "Math Error: Cannot divide by zero"
  - Square root of negative number → throw Error "Math Error: Cannot calculate square root of negative number"
  - Power operation with base 0 and negative exponent → throw Error "Math Error: 0 cannot be raised to negative power"
- **Range Errors:**
  - Result exceeds Number.MAX_SAFE_INTEGER → throw Error "Result too large to calculate accurately"
  - Result is Infinity → throw Error "Result is infinity"

3. **Error Display:**
   - Use try-catch-finally blocks
   - Display error.name and error.message in red color
   - Log all errors to an "Error Log" section with timestamp
   - finally block should always display "Calculation attempt completed"
4. **Global Error Handler:**
   - Implement window.onerror to catch any unexpected errors
   - Log these to console with full details

Task 9: Math Operations - Random Number Games

Game 1: Guess the Number

- Generate random number between 1-100
- Give user 7 attempts to guess
- After each guess, tell them if they're too high or too low
- Show distance from target using Math.abs()
- Calculate score based on number of attempts

Game 2: Dice Rolling Simulator

- Simulate rolling 2 dice
- Display results and sum
- Keep statistics of all rolls (track frequency of each number 1-6)
- Display which number appears most frequently after 20 rolls
- Show probability calculations (actual frequency vs expected)

Game 3: Random Math Quiz

- Generate random math problems:
  - Addition: two numbers between 1-100
  - Multiplication: two numbers between 1-20
  - Rounding: give decimal number, ask to round to nearest integer
  - Max/Min: give 5 numbers, ask for maximum or minimum
- Track correct/incorrect answers
- Calculate percentage score
- Display time taken for each question

Requirements:

- Use appropriate Math methods (random, floor, ceil, round, abs, max, min)
- Implement proper scoring systems
- Display statistics after game completion
- Add "Play Again" functionality

Solve day 3, day 6 and day 7 in : Hacker Rank